SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: MLA03

COURSE CODE: Reinforcement learning for Environment and

Agent COURSE OUTCOME COVERED

CO4: Evaluate model-based reinforcement learning approaches for prediction,

planning, and policy optimization. (BL3, BL4, BL5)

Assessment Tool Weightage: 30%

Dr G. A. SENTHIL QUESTIONS: 10 Total Marks: 50

3. Deep Q-Networks for Intelligent Decision-Making

Deep Q-Network (DQN) is a Deep Reinforcement Learning algorithm that combines Q learning with a neural network. It is used to learn the best action for a given state by estimating the Q-value of each possible action.

DQN is especially useful when the state space is large and it is difficult to store a Q-table. Instead of maintaining a table, a neural network approximates the Q-function and helps the agent make intelligent decisions.

1. DQN Architecture

Definition

DQN uses a neural network to approximate the action-value function Q(s,a). The network receives the current state as input and produces Q-values for the available actions as output.

The action with the highest Q-value is normally selected by the agent.

Architecture

Environment

↓

Current State

↓

Deep Neural Network

↓

Q-values for Available Actions

↓

Select Best Action

↓

Environment

↓

Reward + Next State

Working

1. The agent observes the current state of the environment. 2.

The current state is given as input to the deep neural network.

3. The network calculates the Q-value for every possible

action.

4. The agent selects an action using an ε-greedy strategy. With probability ε it explores a random action, otherwise it chooses the action with the highest Q-value.

5. The environment provides a reward and the next state.

6. The experience is stored for training.

7. The neural network is updated using the Q-learning target.

8. The process is repeated until the agent learns a good decision-making policy.

Q-Learning Update

The basic Q-learning target used by DQN is:

$$\text{Target} = r + \gamma \max Q(s', a')$$

where:

- r = reward received from the environment
- γ = discount factor
- s' = next state
- a' = possible next action

Advantages

- Can handle large state spaces.
- Uses a neural network instead of a large Q-table.
- Can learn complex decision-making policies.
- Useful for game playing, robotics and autonomous systems.
- Can learn directly from interactions with the environment.

Disadvantages

- Training can be computationally expensive.
- Learning may become unstable without suitable techniques.
- Requires careful selection of

hyperparameters.

- May require many training experiences.
- Basic DQN can suffer from overestimation of

Q-values. **2. Experience Replay**

Definition

Experience Replay is a technique used in DQN to store the agent's previous experiences and reuse them during training. An experience is commonly represented as (state, action, reward, next state, done).

Working

Agent interacts with Environment

↓

Experience (s, a, r, s′, done)

↓

Replay Memory

↓

Random Mini-Batch

↓

Neural Network Training

1. The agent interacts with the environment.

2. Each experience is stored in a replay memory.
3. A random mini-batch is sampled from the memory.

4. The sampled experiences are used to calculate training

targets. 5. The neural network weights are updated.

6. The process continues as new experiences are collected.

Importance

- Breaks the strong correlation between consecutive experiences.
- Allows old experiences to be reused multiple times.
- Improves data efficiency.
- Makes DQN training more stable.

3. Target Networks

Definition

A Target Network is a separate neural network used to calculate the target Q-values during DQN training. It has the same architecture as the main network but its parameters are updated less frequently.

Working

Current State

↓

Main Network → Predicted Q-value

↓

Target Network → Target Q-value

↓

Calculate Loss

↓

Update Main Network

1. The main network estimates the current Q-value.

2. The target network estimates the Q-value of the next

state. 3. A stable target is calculated using the target

network.

4. The loss between the predicted value and target value is

calculated. 5. The main network is trained using this loss.

6. After a fixed number of steps, the target network is updated from the main network.

Importance

- Reduces rapid changes in the training target.
- Improves learning stability.
- Reduces oscillations during training.
- Helps DQN converge more reliably.

4. Industrial Applications in Robotics and Autonomous Systems

Robotics

DQN can be used by robots to learn actions through repeated interaction with an environment. The robot observes its state, selects an action, receives a reward and improves its policy over time.

Example: Robot Navigation

Robot Position + Sensor Information

↓

DQN

↓

Move Forward / Turn Left / Turn Right / Stop

↓
Reward

↓

Policy Improvement

Autonomous Vehicles

DQN can support decision-making tasks such as lane selection, obstacle avoidance and route-level action selection. The state can contain information from sensors, vehicle speed, road position and nearby objects.

- Positive reward for safe movement and reaching the destination.
- Negative reward for collisions or unsafe actions.
- The agent learns safer decisions through repeated training.

Industrial Applications

- Warehouse robot navigation.
- Robot path planning.
- Autonomous vehicle decision-making.
- Drone navigation and obstacle avoidance.
- Industrial robotic control.
- Resource allocation and scheduling in automated systems.

Summary

DQN improves traditional Q-learning by using a deep neural network. Experience Replay improves data efficiency and reduces correlation between experiences, while Target Networks stabilize the training process. Together, these techniques make DQN suitable for complex decision making tasks in robotics and autonomous systems.

4. Comparative Analysis of Advanced Deep Q-Learning Algorithms

Advanced Deep Q-Learning algorithms improve the basic DQN algorithm by addressing problems such as overestimation, inefficient representation of state values and inefficient use of training experiences.

Three important improvements are Double DQN (DDQN), Dueling DQN and Prioritized Experience Replay (PER).

1. Double DQN (DDQN)

Definition

Double DQN is an improvement over DQN designed mainly to reduce the overestimation of action values. It separates action selection from action evaluation.

Working

1. The online network selects the best action for the next

state. 2. The target network evaluates the selected action.

3. The resulting target value is used to train the online network.

4. This separation reduces the tendency to overestimate

Q-values. DDQN Target:

Target = r + γ Q_target(s′, argmax Q_online(s′, a′))

Advantages

- Reduces overestimation bias.
- Provides more reliable Q-value estimates.
- Can improve learning performance compared with basic DQN.
- Uses the existing online and target network structure.

Applications

- Game playing.
- Robot decision-making.
- Autonomous navigation.

- Sequential decision-making problems.

2. Dueling DQN

Definition

Dueling DQN is an architecture improvement in which the neural network separates the estimation of the state value from the advantage of each action.

Architecture

State

↓

Shared Neural Network

↙ ↘

State Value V(s) Action Advantage A(s,a)

↘ ↙

Combine Value + Advantage

↓

Q-values

↓

Select Action

Working

1. The agent observes the current state.

2. The state is processed by shared neural network layers.

3. One stream estimates how valuable the current state is.

4. Another stream estimates the advantage of each

action. 5. The two streams are combined to calculate

Q-values.

6. The action with the highest useful Q-value is selected.

Advantages

- Separates state value and action advantage.
- Can learn useful state information even when actions have similar effects.
- Improves representation of action values.
- Can improve learning efficiency in some environments.

Applications

- Robotics.
- Navigation systems.
- Game environments.
- Autonomous decision-making.

3. Prioritized Experience Replay (PER)

Definition

Prioritized Experience Replay improves the standard replay memory by giving higher priority to experiences that are more useful for learning. Experiences with larger learning errors are more likely to be selected.

Working

Agent Experience

↓

Calculate TD Error

↓

Assign Priority

↓

Prioritized Replay Memory

↓

Sample Important Experiences

↓

Update Network

1. The agent collects experiences from the environment. 2. A temporal-difference (TD) error is calculated for each experience. 3. Experiences with larger errors receive higher priority. 4. Important experiences are sampled more frequently. 5. The network learns from these experiences.

6. The priorities are updated as learning progresses.

Advantages

- Uses training experiences more efficiently.
- Focuses learning on experiences with larger errors.
- Can speed up learning.
- Can improve sample efficiency compared with uniform replay.

Disadvantages

- Adds additional computational complexity.
- Requires priority calculation and updating.
- Sampling bias must be considered and corrected.

4. Performance Comparison Using Benchmark Environments

Benchmark Environments

Advanced DQN algorithms can be evaluated using standard benchmark environments such as CartPole, LunarLander, Atari games and other reinforcement learning environments.

Comparison

DQN

- Basic deep Q-learning approach.
- Uses experience replay and target networks.
- Can suffer from Q-value overestimation.
- Provides a baseline for comparison.

Double DQN (DDQN)

- Reduces overestimation bias.
- Often gives more stable Q-value estimates.
- Useful when accurate action evaluation is

important. Dueling DQN

- Separates state value and action advantage.
- Can learn efficiently when many actions have similar

effects. Improves the network's representation of useful state information. Prioritized Experience Replay (PER)

- Samples important experiences more frequently.
- Improves use of replay data.
- Can improve sample efficiency and learning speed.

Performance Measures

- Average episode reward.

- Average episode length.
- Convergence speed.
- Learning stability.
- Sample efficiency.
- Final performance after training.

Example Comparative Table

Algorithm	Main Improvement	Main Problem Addressed	Learning Focus	Expected Benefit
DQN	Deep Q-network	Large state spaces	All replay samples	Strong baseline
DDQN	Separate action selection and evaluation	Q-value overestimation	Accurate action evaluation	More reliable values
Dueling DQN	Value and advantage streams	Poor state/action representation	State value + action advantage	Better representation
PER	Prioritized replay	Inefficient uniform sampling	High-error experiences	Better sample efficiency

Overall Comparison

Among these methods, DDQN mainly improves the accuracy of Q-value estimation, Dueling DQN improves the neural network architecture, and PER improves the way training experiences are selected. They can also be combined to form stronger DQN-based agents.

Conclusion

Advanced Deep Q-Learning algorithms improve the limitations of basic DQN in different ways. DDQN reduces overestimation, Dueling DQN provides a better value and advantage representation, and PER concentrates learning on important experiences. Their performance can be compared using benchmark environments by measuring reward, convergence, stability and sample efficiency. These improvements are useful for intelligent decision-making in games, robotics and autonomous systems.